

Decision Trees, Random Forests, and Boosting

Decision Trees

Decision trees are used for predictive modelling. They can be categorized into:

- **Classification Trees:** Used to classify data into categories.
- **Regression Trees:** Used to predict numeric values.

Structure of a Decision Tree:

1. **Root Node:** The topmost node, from which the tree begins branching.
2. **Branches:** Pathways connecting nodes, with arrows pointing to them and they have arrows pointing away from them.
3. **Leaf Nodes:** The endpoints of the tree. Leaves have arrows pointing to them but none pointing away.

How to Build a Decision Tree:

1. Select a column from the dataset as the **root node**. This is chosen based on how well it predicts the required outcome.
2. Split nodes into branches and further divide them until **impurity** is minimized. When no further useful splits can be made, a **leaf node** is created.

Measuring Impurity (Gini Impurity):

- Leaves containing a mixture of outcomes are considered **impure**.
- The impurity of a leaf is calculated using the formula:

$$G = 1 - (\text{the probability of "yes"})^2 - (\text{the probability of "no"})^2$$

Whichever node has the lowest Gini impurity, shows that its leaves have the lowest impurity. It can be an ideal choice for root node.

- The overall Gini impurity is the weighted average of the impurities of all leaves. Nodes with the lowest impurity are preferred for splitting.

Overfitting:

- Overfitting happens when a tree performs well on training data but poorly on unseen data.
- To avoid overfitting, ensure that each split significantly reduces impurity.

Random Forests

Random forests are built from multiple decision trees to improve prediction accuracy. They combine the **simplicity of decision trees** with **flexibility**, making them better suited for new data.

How to Build a Random Forest:

1. **Bootstrap Dataset:** Create random subsets of the dataset (with replacement). Each subset may contain duplicates of samples.
2. Build decision trees using these subsets, ensuring that at each step only a **random subset of variables** is considered.
3. Pass the observation through all trees and determine the final output by **majority voting**.

Key Concepts:

- **Bagging:** Aggregating the results of bootstrapped datasets for better predictions.
- **Out-of-Bag (OOB) Dataset:** The samples excluded during bootstrapping. These samples are passed through the forest to calculate the **OOB error**—a measure of accuracy.

Handling Missing Data:

1. **In Original Dataset:**
 - Replace missing Boolean values with the most common value.
 - Replace missing numeric values with the mean.
 - Refine by identifying **similar samples** (those ending in the same leaf node).
2. **In New Data:**
 - Create two versions of the dataset: one with the missing value as true, the other as false.
 - Pass both through the trees and select the most consistent result.

Boosting

Boosting is an ensemble method that combines multiple **weak learners** (models slightly better than random guessing) into a **strong learner**. It focuses on correcting errors from earlier iterations, enhancing accuracy.

Adaboost:

- Adaboost uses **stumps** (trees with one node and two leaves) as weak learners.

- While random forests give equal voting weight to all trees, Adaboost assigns more importance (or "say") to stumps that perform better.

Key Features:

1. **Order Matters:** Each stump is influenced by the mistakes of the previous one.
2. **Weighted Voting:** Stumps with better accuracy have a higher impact on the final decision.

How Adaboost Works:

1. Choose the first stump using the Gini index. The stump with the lowest Gini impurity is selected.
2. Calculate the **total error** for the stump:
 - Total error is the sum of the weights of incorrectly classified samples. It ranges from 0 (perfect stump) to 1 (entirely incorrect stump).
3. Determine the "**amount of say**" for the stump:
4. Amount of Say = $\frac{1}{2} \log\left(\frac{1 - \text{total error}}{\text{total error}}\right)$

Comparison of Random Forests and Random forest made with Adaboost:

- **Random Forest:**
 - Trees are independent, and the order of creation doesn't matter.
 - All trees have equal voting weight.
- **Adaboost:**
 - Stumps are created sequentially, with each accounting for the errors of the previous one.
 - Voting weights depend on the accuracy of the stumps.